Guía de Disertación: Blue Team Defense Center

> **Audiencia:** Profesor evaluador y compañeros de clase de Ciberseguridad.
> **Integrantes del grupo:** 4 Estudiantes.
> **Duración Objetivo:** 10 a 15 Minutos.
> **Tema Asignado:** Blue Team (Defensa).

🧑‍💻 ESTUDIANTE 1: Concepto y Objetivo (Diapositiva 1 y 2)

1. 🧠 ¿Qué es nuestro equipo? (Blue Team)

Nosotros representamos al **Blue Team** (Equipo Azul). En ciberseguridad, somos el equipo de seguridad defensiva interna de la organización. A diferencia del Red Team que se encarga de atacar para buscar vulnerabilidades, nuestra mentalidad es actuar como el escudo protector corporativo de la infraestructura tecnológica.

¿Cuál es el objetivo principal?

Nuestro objetivo fundamental es proteger y garantizar la **Confidencialidad, Integridad y Disponibilidad** (La Tríada de la Seguridad) de los activos de la empresa. Lo hacemos identificando fallas en las aplicaciones, monitorizando redes en tiempo real y desarrollando parches ('hardening') para evitar que un atacante logre extraer datos.

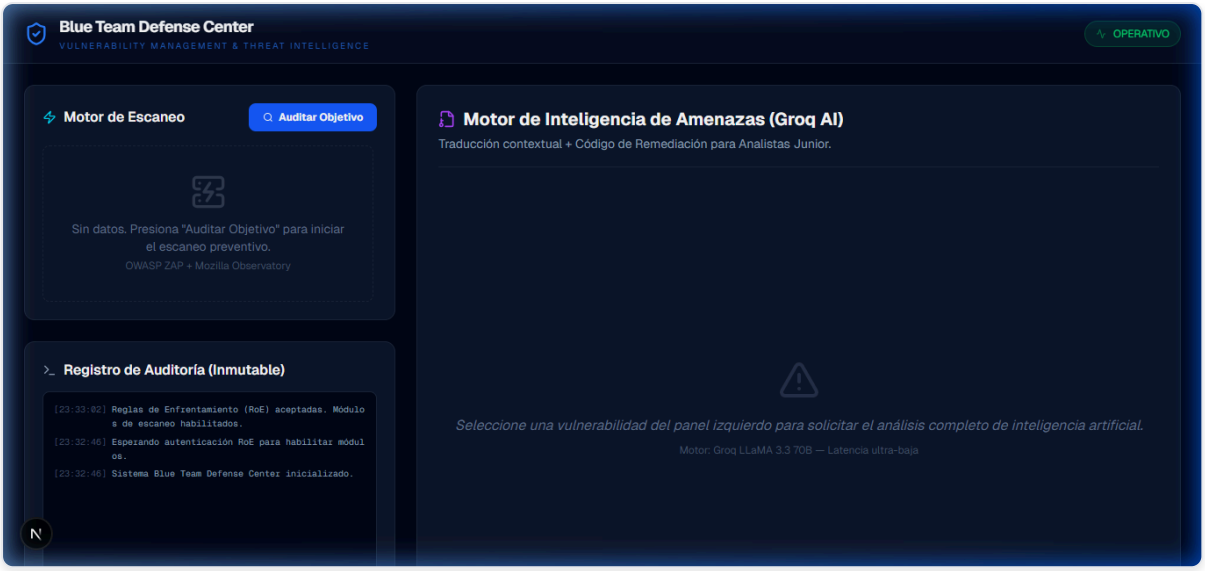

Figura 1: Vista general de nuestro panel de control defensivo.

🧑‍💻 ESTUDIANTE 2: Funciones y Herramientas (Diapositiva 3 y 4)

2. ⚙️ Funciones Principales y Etapas de Trabajo

¿En qué etapa trabajamos? Prácticamente en las 3 etapas críticas:

- **Prevención:** Configuramos Firewalls y políticas de red antes de que ocurra un ataque.
- **Detección:** Estamos constantemente monitorizando alertas de tráfico anómalo.
- **Respuesta:** Cuando detectamos una inyección de código, aplicamos planes de contingencia (playbooks) para expulsar al intruso y tapar el agujero inmediatamente.

3. 🛠️ Herramientas Reales Utilizadas

Como exige el estándar de la industria, el Blue Team se apoya en 3 herramientas fundamentales (las cuales integramos conceptualmente en nuestro proyecto):

- **SIEM (Ej: Splunk / Elastic Security):** Herramienta vital de detección. Recopila millones de registros (logs) de toda la empresa y los correlaciona para dispararnos alertas en el Dashboard cuando alguien intenta algo raro.
- **Mozilla Observatory:** Nos sirve en la etapa de Prevención. Audita empíricamente nuestro sitio web para ver si el candado HTTPS y las políticas

de seguridad (HSTS, CSP) están firmes contra ataques como el Man-In-The-Middle.

- **Escáneres y Crawlers (Ej: OWASP ZAP / Nessus):** Herramienta que mapea toda nuestra red buscando puertas traseras dejadas por error o configuraciones por defecto peligrosas.

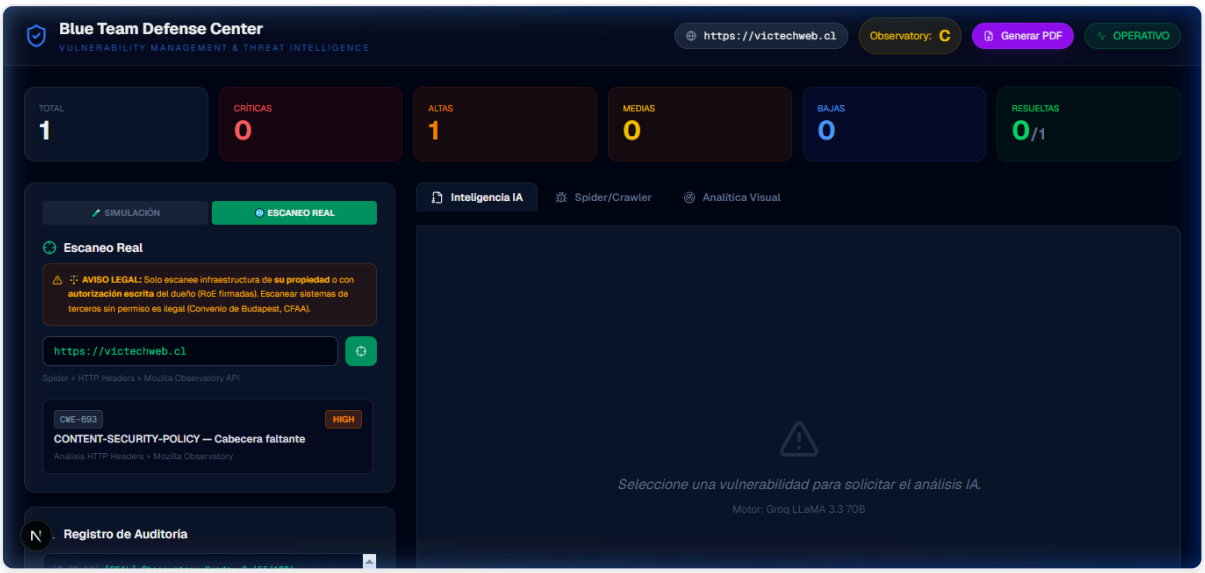

Figura 2: Análisis mediante la herramienta real Mozilla Observatory integrada en nuestro sistema.

🧑‍💻 ESTUDIANTE 3: El Ejemplo Práctico (Diapositiva 5 y 6)

4. 🔍 Ejemplo Práctico: Caso de Inyección SQL

Vamos a simular un caso real con el software que hemos construido. Supongamos que un atacante del Red Team (o un hacker real) intentó acceder al panel de nuestra empresa ficticia buscando una brecha.

- **Paso 1 (Detección):** Nuestro Splunk/SIEM lanza una alerta en nuestro Panel ("SQL Injection en Formulario de Login"). Un ciberdelincuente inyectó el código malicioso `admin' OR '1'='1'`.
- **Paso 2 (Análisis):** En vez de entrar en pánico, nuestro analista de Blue Team da clic en la alerta. Se despliega una inmensa consola detallando que el atacante intentó evadir la autenticación.

Figura 3: Alertas rojas Críticas detectadas en el sistema de defensa.

- **Paso 3 (Respuesta y Remediación Inteligente):** Aquí usamos tecnología de punta. Para responder en tiempo récord, enviamos esta telemetría a una Inteligencia Artificial (LLaMA 3.3, actuando como nuestro Asistente SOAR de respuesta automatizada). La IA nos devuelve exactamente el parche que el desarrollador debe copiar en el servidor (Consultas parametrizadas) para detener completamente el ataque en menos de 2 minutos.

Blue Team Defense Center
VULNERABILITY MANAGEMENT & THREAT INTELLIGENCE

Objetivo: `http://localhost:3001` OPERATIVO

TOTAL ALERTAS	CRÍTICAS	ALTAS	MEDIAS	RESUELTAS
7	1	2	3	0/7

Motor de Escaneo — Auditar Objetivo

- CWE-89 — CRITICAL — SQL Injection en formulario de... — OWASP ZAP (Active Scan) A03:2021 – Injection
- CWE-79 — HIGH — Cross-Site Scripting (XSS... — OWASP ZAP (Active Scan) A03:2021 – Injection (XSS)
- CWE-693 — HIGH — Content-Security-Policy (CSP)... — A05:2021 – Security Misconfiguration
- CWE-16 — MEDIUM — X-Content-Type-Options Header... — Mozilla A05:2021 – Security Misconfiguration

1 Issue

Registro de Auditoría (Inmutable)

```
[22:39:52] Motor Groq LLaMA 3.3: Traducción recibida para CWE-89.
[22:39:48] Solicitando análisis de inteligencia para CWE-89 (SQL Injection en formulario de Login)
[22:39:16] Escaneo finalizado: 7 vulnerabilidades detectadas. [Criticas: 1, Altas: 2, Medias: 3, Bajas: 1]
[22:39:16] Objetivo escaneado: http://localhost:3001
```

Motor de Inteligencia de Amenazas (Groq AI)

Traducción contextual + Código de Remediación para Analistas Junior.

CWE-89 — SQL Injection en formulario de Login

CRITICAL

Categoría OWASP: A03:2021 – Injection
URL Afectada: `http://localhost:3001/api/login`
Evidencia: Parámetro 'username' no sanitizado. Respuesta del servidor devuelve token de autenticación con payload malicioso.

🔍 ¿Qué significa esta vulnerabilidad?

La vulnerabilidad CWE-89, también conocida como Inyección SQL, se produce cuando una aplicación web construye consultas SQL mediante la concatenación directa de cadenas sin parametrización. Esto permite a un atacante inyectar código SQL malicioso para manipular la base de datos. Imagina que tienes un formulario de login donde se pide el nombre de usuario y la contraseña. Si la aplicación no valida ni sanitiza la entrada, un atacante podría introducir algo como "admin' OR '1'='1" en el campo de nombre de usuario. La aplicación, al no tener una validación adecuada, construiría una consulta SQL que incluiría esta entrada, lo que permitiría al atacante acceder al panel administrativo sin credenciales válidas. Es como si un ladrón encontrara una llave maestra que abre todas las puertas de una casa.

⚠️ Impacto en la Organización

El impacto de esta vulnerabilidad es significativo. Si un atacante la explota, podría acceder a datos sensibles, como información de clientes, credenciales de acceso, o incluso tomar el control de la base de datos. Esto podría llevar a una pérdida de confianza por parte de los clientes, ya que se sentirían vulnerables y expuestos. El equipo de TI se vería sometido a un estrés operativo alto, ya que tendrían que trabajar bajo presión para mitigar el daño y restaurar la seguridad. Además, la empresa podría enfrentar consecuencias legales y reputacionales, como multas, demandas, y daños a su imagen pública.

🛡️ Código de Remediación (Parche del Blue Team)

Para remediar esta vulnerabilidad, se deben utilizar consultas parametrizadas o preparadas. A continuación, se muestra un ejemplo de cómo hacerlo en Next.js/Node.js y Nginx:

Next.js / Node.js

```
// Importar el paquete mysql2
const mysql = require('mysql2/promise');

// Configurar la conexión a la base de datos
const db = await mysql.createConnection({
  host: 'localhost',
  user: 'usuario',
  password: 'contraseña',
  database: 'basedatos'
});

// Utilizar consultas parametrizadas
const query = 'SELECT * FROM usuarios WHERE nombre = ? AND contraseña
```

```
const [results] = await db.execute(query, [nombre, contraseña]);
```

Nginx

No es necesario realizar cambios en la configuración de Nginx para remediar esta vulnerabilidad, ya que se trata de un problema de aplicación. Sin embargo, se puede configurar Nginx para que bloquee solicitudes maliciosas utilizando el módulo `**nginx-http-lua**` y una regla como la siguiente:

```
http {
    ...
    lua_shared_dict blocklist 10m;
    init_by_lua_block {
        blocklist = {}
    }
    ...
    server {
        ...
        location /api/login {
            ...
            lua_need_request_body on;
            access_by_lua_block {
                local body = ngx.req.get_body_data()
                if body then
                    local nombre = ngx.re.match(body, "nombre=([^&]+)"
                    local contraseña = ngx.re.match(body, "contraseña=
                    if nombre and contraseña then
                        local query = "SELECT * FROM usuarios WHERE n
                        -- Verificar si la consulta es válida
                        if query then
                            -- Permitir la solicitud
                            return
                        else
                            -- Bloquear la solicitud
                            ngx.exit(403)
                        end
                    end
                end
            }
        }
    }
}
```

Ten en cuenta que esta es solo una medida de seguridad adicional y no reemplaza la necesidad de utilizar consultas parametrizadas en la aplicación.

📋 Verificación Post-Parche

Para verificar que el parche fue aplicado correctamente, se pueden realizar las siguientes pruebas:

1. Utilizar `**curl**` para enviar una solicitud de login con un nombre de usuario y contraseña válidos:

```
curl -X POST -H "Content-Type: application/x-www-form-urlencoded" -d '
```

2. Verificar que la respuesta sea exitosa y que se devuelva el token de acceso o la información del usuario.
3. Intentar inyectar código SQL malicioso en el campo de nombre de usuario:

```
curl -X POST -H "Content-Type: application/x-www-form-urlencoded" -d '
```

4. Verificar que la respuesta sea un error de autenticación y que no se devuelva el token de acceso o la información del usuario.
5. Repetir las pruebas con diferentes combinaciones de nombres de usuario y contraseñas para asegurarse de que la vulnerabilidad esté completamente cerrada.

Figura 4: Consola de remediación, donde neutralizamos el ataque usando código defensivo programado por IA.

🧑‍💻 ESTUDIANTE 4: Relación entre Equipos y Cierre (Diapositiva 7)

5. 🔄 Interacción con los otros Equipos

Para terminar, es vital entender que el Blue Team no puede existir de forma aislada. Si solo defendemos a ciegas, nos estancaremos en el tiempo.

- **Relación con Red Team:** De ellos aprendemos los "TTPs" (Tácticas, Técnicas y Procedimientos). Cuando el Red Team nos 'hackea' exitosamente en un simulacro, nosotros estudiamos exactamente los pasos que ellos hicieron para configurar nuestros Firewalls (WAF) y que nunca más vuelva a suceder.
- **Aparición del Purple Team:** Esta relación tan estrecha fomenta lo que llamamos el modelo Purple Team. Un trabajo colaborativo continuo donde no hay rivalidad. El atacante avisa qué va a intentar y el defensor mejora sus herramientas defensivas en tiempo real.

Conclusión

El trabajo colaborativo garantiza que los parches no se apliquen de forma "reactiva" cuando ya nos robaron los datos, sino de forma "proactiva". La ciberseguridad defensiva no es un departamento solitario, es una cultura de mejora continua. Muchas gracias.